Student Name: DO VU KHANH QUYNH - Student ID: 105029408

Music Player Program Report (Custom Code Project)

1. Introduction

In this project, I used Python together with the Pygame library to build a simple music player. The main idea of my program is to let users browse albums, choose songs, and control playback using a graphical interface.

While developing this program, I focused on applying what I learned in class, such as using functions, lists, and structured programming. I also tried to organize my code into multiple files to make it easier to understand and manage.

To be honest, this part took me quite a while to figure out. At first, my program had a lot of bugs, especially when switching between screens or playing different songs. I had to test many times and sometimes restart everything just to understand what went wrong.

2. Program Overview

The program provides the following features:

- Display a library of albums with images
- View detailed album information
- Select and play songs
- Playback controls (play, pause, next, previous)
- Volume control using a slider
- Progress bar showing song duration
- Repeat mode toggle

The system is implemented using a real-time event loop provided by Pygame.

3. Functional Decomposition

The program is divided into multiple functions across different modules, each responsible for a specific task.

Input Handling

- handle_mouse_click()
  Handles all user interactions such as clicking albums, tracks, and control buttons.

Playback Logic

- play_track()
  Loads and plays the selected song.
- toggle_pause()
  Controls pausing and resuming playback.
- update_player()
  Handles repeat functionality and playback timing.

Rendering (UI)

Custom Code D/HD

- draw()
  Renders the main album library screen.
- draw_album_detail()
  Displays album details and track list.
- draw_controls()
  Draws playback buttons and volume slider.
- draw_progress_bar()
  Displays the current playback progress.

This decomposition improves readability and makes debugging easier.

4. Data Structures

The program uses both arrays (lists) and records (classes):

Arrays (Lists)

- List of albums
- List of tracks within each album

Records (Classes)

- Album class: stores artist name, album name, release date, and tracks
- Track class: stores song name and file location

State Management

An AppState class is used to store all shared data such as:

- Current screen
- Selected album
- Current song
- Playback status
- Volume level

This replaces global variables and improves code structure and maintainability.

5. Structured Programming

The program follows structured programming principles:

Sequence

The program executes in a logical order:

- Initialize system
- Load data
- Enter main loop
- Handle input → update → render

Selection

Conditional statements are used throughout the program:

- Detect which screen is active
- Check user input

- Control playback states

Repetition

A continuous game loop (while True) is used to:

- Process events
- Update the program state
- Render the interface

6. Design and Modularity

The project is organized into multiple modules:

- main.py: entry point of the program
- song.py: main application loop
- ui_controls.py: user interaction logic
- player_logic.py: music playback logic
- app_state.py: shared state management
- screens/: rendering functions

This modular design separates responsibilities and makes the program easier to extend and maintain.

7. Features and Enhancements

The application includes several advanced features:

- Interactive GUI using Pygame
- Real-time playback controls
- Dynamic progress bar
- Volume adjustment
- Repeat mode functionality

These features demonstrate a higher level of program complexity beyond basic requirements.

8. Challenges and Solutions

One difficulty I had was managing variables across different parts of the program. At first, I used global variables, but it quickly became confusing and hard to debug.

To fix this, I created an AppState class to store all important data in one place. This made the program easier to manage and reduced unexpected errors.

There were also times when the buttons did not respond properly, even though the code looked correct. This was quite frustrating, and I had to carefully check the mouse position and click areas step by step.

9. Reflection

I learned more about the following through this project:

- Pygame-based event-driven programming
- Modular design and functional decomposition
- Program state management without global variables
- Creating graphics programs that are interactive

Custom Code D/HD

Student Name: DO VU KHANH QUYNH - Student ID: 105029408
I would enhance the program with features like playlist management, search capabilities, and improved user interface design if I had more time.

Overall, I think I made several mistakes during the development process, but that also helped me understand the program better. If I do this project again, I would probably design it more carefully from the beginning instead of fixing problems later.

10. Conclusion

In conclusion, this project helped me understand how to build an interactive program using Python and Pygame. I was able to apply different programming concepts and improve my coding skills.

Overall, I think this program works well and can be improved further in the future.